digital electronics syllabus summary

nithilan ramesh

November 15, 2025

Contents

1 Module 1: Introduction to Logic Circuits and Optimization (Unit 1)

- **End-Term Weightage:** 20 marks
- **CO Mapping:** CO1 (10 marks) + CO2 (10 marks)
- **Focus:** Realizing expressions and Minimization techniques.

1.1 1.1 Introduction to Logic Circuits

1.1.1 Variables and Functions

- Definition of Boolean Algebra (variables are 0 or 1).
- Representations of Digital Design using Switches.
- Logic expression/Logic Function $L(x) = x$.
- Realizing logic functions using switches connected in series (logical AND) and parallel (logical OR).
- Analyzing complex series-parallel switch connections.

1.1.2 Inversion

- The Complement Operation (NOT operator, denoted by a bar over the variable).
- Implementation of an inverting circuit using switches.

1.1.3 Truth Tables

- Useful aid for providing information about a logic function.
- Truth tables for AND, OR, and 3-input variables.

1.1.4 Logic Gates and Networks

- Implementation of logic functions using Logic Gates (one or more inputs, one output).
- Graphical symbols for basic gates (AND, OR, NOT).
- Implementation of larger circuits by Network of Gates.
- Analysis Process Determining the function performed by an existing network.
- Synthesis Process Designing a new network to implement desired functional behavior.

1.2 1.2 Boolean Algebra and Synthesis

1.2.1 Boolean Algebra

- Axioms, Single Variable Theorems, Two and Three variable Properties.
- **Principle of Duality:** Obtaining the dual by replacing + with $\cdot$ and 0s with 1s (and vice versa).
- DeMorgan's Theorem (Proof using Truth Table).
- Proof of Absorption Law ($X + XY = X$).
- Proof of Consensus Theorem ($XY + X'Z + YZ = XY + X'Z$).

1.2.2 Synthesis using gates

- Sum of Products (SOP) Canonical form using Minterms.
- Product of Sum (POS) Canonical Form using Maxterms (complement of minterms).
- Reducing canonical expressions using Boolean Algebra theorems.

1.2.3 Design examples

- Simple problem synthesis using truth table and Canonical SOP/POS forms.
- Cost comparison between Canonical expression circuit realization and reduced circuit realization (cost = number of gates + total number of inputs).

1.3 1.3 Optimized Implementation of Logic Functions

1.3.1 Optimized implementation of logic functions

- Introduction to **Exclusive-OR (XOR)** and **Exclusive-NOR (XNOR)** Gates.
- XOR output is high when an odd number of inputs are high.
- XNOR output is high when an even number of inputs are high (or all inputs are the same).

1.3.2 Universal Gates (NAND and NOR)

- Any other logic gate can be implemented using only NAND or only NOR gates.
- Function Implementation using NAND Gates Only (SOP form/NAND-NAND network realization).
- Function Implementation using NOR Gates Only (POS form/OR-AND network/NOR-NOR network realization).

1.3.3 Karnaugh map (K-Map)

- A systematic, pictorial method for manually deriving minimum-cost implementations.
- Structure of Two-Variable, Three-Variable, and Four-Variable K-Maps.
- The use of Gray code sequence in K-Maps.
- Combining adjacent cells (pairs, quads, octas) to reduce the number of literals in the product term.
- Extension to Five-Variable K-Map (using 2 four-variable maps).

1.3.4 Strategy for minimization

- Objective: Finding the minimum cost circuit.
- **Literal:** Each appearance of a variable (complemented or uncomplemented).
- **Implicant:** A product term where the function equals 1.
- **Prime Implicant (PI):** An implicant that cannot be combined into another implicant with fewer literals.
- **Cover:** A collection of implicants that accounts for all valuations where the function is 1.
- **Essential Prime Implicant (EPI):** A prime implicant that includes a minterm not covered by any other prime implicant.
- **Steps for minimum-cost circuit:** 1. Generate all PIs. 2. Find EPIs. 3. Select non-EPIs to complete the cover based on cost.

1.3.5 Minimization of product of sums forms

- Uses the principle of duality, combining maxterms (where $f = 0$) into the largest possible sum terms.
- Comparing the minimum cost of SOP versus POS implementation solutions.

1.3.6 Incompletely specified functions

- Use of **don't-care conditions (D)** where certain input valuations never occur.
- Assigning don't cares (0 or 1) to facilitate forming the largest possible groups of 1s or 0s to achieve the lowest-cost prime implicants in SOP or POS forms.

1.3.7 Multiple output circuits

- Implementing multiple functions using a single circuit to reduce overall cost by sharing gates that implement identical product terms.

1.3.8 Tabular method for minimization

- Also known as the **Quine-McCluskey method**, used when the number of variables exceeds five or six, as K-maps become difficult.
- Algorithm involves listing minterms, arranging by number of 1s, comparing and combining adjacent terms, and selecting the minimum cover from the set of prime implicants.

---

2 Module 2: Combinational Circuit Building Blocks (Unit 2)

- **End-Term Weightage:** 30 marks
- **CO Mapping:** CO3 (30 marks)
- **Focus:** Design of Combinational circuits.

2.1 2.1 Number Representation and Arithmetic Circuits

2.1.1 Number Representation

- Types of Positional Representation (Decimal-Base 10, Binary-Base 2, Octal-Base 8, Hexadecimal-Base 16).
- Conversions (Binary to Decimal, Decimal to Binary, Octal to Binary/Decimal using 4-2-1 code, Hexadecimal to Binary/Decimal using 8-4-2-1 code).

2.1.2 Addition of unsigned numbers

- Unsigned Binary Addition and Subtraction.
- **Half-Adder:** Circuit for adding two bits.
- **Full-Adder:** Circuit for adding three bits (X_i, Y_i, and carry-in C_i).
- Decomposed Full Adder using two Half Adders.
- **Ripple Carry Adder (RCA):** Structure used for multi-bit addition by propagating carry sequentially.
- **Timing delay in RCA:** The sum is available after a delay of $n\Delta t$ for an n-bit adder due to carry propagation delay.

2.1.3 Signed numbers

- Methods for negative number representation: Sign-and-magnitude, 1's complement, and 2's complement.
- Drawbacks of Sign-and-magnitude (complexity, not used in computers).
- **1's Complement:** Obtained by complementing all bits of the positive number ($K = (2^n - 1) - P$).
- **2's Complement:** Obtained by subtracting the positive number from 2^n ($K = 2^n - P$), or 1's complement plus one.
- **2's Complement Addition and Subtraction:** Most suitable method as subtraction can be performed using the adder itself (no extra carry addition overhead).

2.1.4 Fast adders

- Addressing the carry propagation delay in RCA.
- **Carry-Lookahead Adder (CLA):** Evaluates the carry-out function quickly using a two-level AND-OR circuit structure.
- CLA significantly reduces the critical path delay compared to RCA.

2.1.5 Arithmetic Overflow

- Occurs if the result of signed addition/subtraction does not fit within the n-bit range (-2^{n-1} to $2^{n-1} - 1$).
- Need for circuits to detect overflow.

2.2 2.2 Combinational Building Blocks

2.2.1 Multiplexers (MUX)

- **Definition:** Has N control inputs, 2^n data inputs, and 1 output.
- **Function:** Routes the selected data input to the output based on control inputs.
- 2-to-1 Multiplexer (Graphical symbol, truth table, SOP circuit).
- MUX as a Universal Gate (can construct AND and NOT gates).
- Building larger MUXes from smaller MUXes (e.g., 4-to-1 MUX from 2-to-1 MUXes).
- Synthesis of logic circuits using MUXes (implementation of complex functions like XOR or Majority using appropriate control and data inputs).

2.2.2 Decoders

- **Definition:** Logic circuit with n inputs and 2^n outputs, used to decode encoded information.
- Only one output is asserted at a time; outputs are one-hot encoded (active-high or active-low).
- Includes an Enable input (En) to disable outputs.
- Examples: 2-to-4 Decoder, 3-to-8 Decoder.
- Construction of larger decoders from smaller decoders (e.g., 3-to-8 decoder from two 2-to-4 decoders) forming a decoder tree.
- Function Implementation using Decoder: Can implement SOP or POS forms of logic functions (e.g., Full Adder implementation).

2.2.3 Encoders

- Performs the reverse operation of a decoder (a 2^n-to-n binary encoder).
- Examples: 4-to-2 binary encoder, 8-to-3 binary encoder (octal-to-binary).
- **Priority Encoder:** Used to ensure only one input is processed if multiple inputs are high.

2.2.4 Code converters

- **BCD to 7-segment Display decoder:** Converts a BCD digit into seven signals (a to g) suitable for driving a display.
- Uses K-map simplification and utilizes don't-care conditions for minimization.

2.2.5 Demultiplexers (De-MUX)

- Performs the reverse operation of a Multiplexer (single input, n selection lines, maximum 2^n outputs).
- The input is connected to one output based on selection lines.
- An n-to-2^n decoder circuit can be used as a 1-to-n demultiplexer.

2.2.6 Arithmetic comparison circuits

---

3 Module 3: Sequential Circuit Building Blocks (Unit 3)

- **End-Term Weightage:** 50 marks
- **CO Mapping:** CO4 (50 marks)
- **Focus:** Design of Sequential circuits.

3.1 3.1 Latches and Flip-Flops

3.1.1 Sequential circuit building blocks

- Sequential circuits include storage elements (flip-flops) and the output depends on the previous state and present inputs.
- Clocks are used to synchronize circuits (synchronous circuits).

3.1.2 Basic latch

- A simple memory element built with cross-coupled NOR or NAND gates.

3.1.3 SR Latch

- Defines Set (S=1, R=0) and Reset (S=0, R=1) states. Has an Invalid state when S=R=1.

3.1.4 Gated SR latch

- Includes a Clock (Clk) input to enable changes only when the clock is active.

3.1.5 Gated D latch

- Data (D) input controls the next state; Q follows D when Clk=1.

3.1.6 Master slave and edge triggered

- **Edge-triggered operation:** Changes occur only at the positive or negative edge of the clock signal.
- **Master-Slave Flip-Flop:** Uses a master latch and a slave latch (e.g., Master Slave D Flip Flop is Negative Edge Triggered).

3.1.7 Auxiliary inputs

- Clear and Preset (can be low-level or high-level active).
- Distinction between Asynchronous Clear (Q changes immediately, regardless of clock) and Synchronous Clear (Q changes only upon the required clock edge).

3.1.8 Timing constraints

- **Setup Time (t_{su})** (D must be stable before clock edge) and **Hold Time (t_h)** (D must remain stable after clock edge). Failure to adhere to these leads to a Metastable state.

3.1.9 D flip-flops

- Characteristic Equation is $Q^+ = D$.

3.1.10 T flip-flop

- Toggles state (Q') when $T = 1$ at the clock edge. Characteristic Equation is $Q^+ = T \oplus Q$.

3.1.11 JK flip-flop

- Master-slave structure that removes the invalid state problem of the SR latch; when J=K=1, the state toggles. Characteristic Equation is $Q^+ = JQ' + K'Q$.

3.1.12 Flip-Flop Conversion

- Steps involving identifying characteristic table of required FF and excitation table of available FF to derive boolean expressions for inputs.

3.2 3.2 Registers and Counters

3.2.1 Registers

- Used as storing elements (e.g., 4-bit Register).

3.2.2 Shift Register

- Shifts information 1 bit right or left; typically implemented as a chain of D flip-flops.
- Requires edge-triggered or master-slave flip-flops (level-sensitive latches are unsuitable).
- Serial Input-Serial Output (SISO).

3.2.3 Parallel-Access Shift Register

- Allows selection between shifting (serial operation) and loading data in parallel using a control signal.
- Types: Series-to-parallel converter and Parallel-to-series converter.

3.2.4 Asynchronous Counters

- Also known as **Ripple Counters**.
- The clock inputs of the flip-flops are connected in cascade.
- Simple to build but slow due to propagation delay (rippling of carries).
- Implemented often using T flip-flops.
- Can be configured as an Up Counter or a Down Counter.

3.2.5 Synchronous Counters

- Faster because all flip-flops are clocked simultaneously.
- Can be implemented using T, JK, or D flip-flops.
- Design steps involve creating a state transition diagram, excitation table, and minimizing input equations using K-Maps.

3.2.6 Ring Counter

- Constructed from a simple shift register where the Q output of the last stage is fed back to the input of the first stage.
- Generates a sequence using a one-hot code (single 1 circulating).

3.2.7 Johnson Counter

- A variation of the ring counter where the inverted output (Q') of the last stage is fed back to the first stage.
- An n-bit counter generates a counting sequence of length $2n$.

3.3 3.3 Synchronous Sequential Circuits Design

3.3.1 Synchronous sequential circuits

- **Synchronous sequential circuits:** Circuits where operations are controlled by a clock signal.
- **Basic design steps:**
  1. Understand specifications.
  2. Draw the state graph (or state diagram).
  3. Construct the state table.
  4. Perform state minimization (if required).
  5. **State assignment problem (Encoding states):** Selecting the number of flip-flops needed.
  6. Create the state-assigned table.
  7. Select the type of Flip-Flop to use.
  8. Determine Flip-Flop input equations and FSM output equations.
  9. Draw the logic diagram.
- **Design of Mealy and Moore state models:**
  - **Mealy state model:** The output depends on both the present state AND the present input.
  - **Moore state model:** The output depends only on the present state.